Prédire les cas de COVID-19 à sept jours

Une chaîne MLOps complète, des données publiques mondiales jusqu'à une API d'inférence conteneurisée, réentraînée automatiquement.

RÉALISÉ PAR

Luvengani Dass
Saadia Szapiro

DONNÉES

Our World in Data
France · 2020 → 2023

MODÈLE

Random Forest
11 variables · horizon J+7

LIVRABLE

4 services Docker
API · MLflow · Airflow

Anticiper une semaine à l’avance, avec ce que l’on sait aujourd’hui

- **La question posée.** À partir de l’historique connu le jour J, estimer le nombre de cas qui sera publié le jour J+7.
- **Pourquoi sept jours.** C’est le délai utile : une semaine suffit pour ajuster une capacité de dépistage, un planning hospitalier ou une campagne de rappel, et reste assez court pour que le passé récent porte de l’information.
- **La contrainte de rigueur.** Aucune variable postérieure au jour J ne doit entrer dans le modèle, sous peine de fuite temporelle — une erreur qui produit d’excellents scores et un modèle inutilisable.

FORMULATION

Régression supervisée sur série temporelle. Une ligne = un jour. La cible est la valeur observée sept jours plus tard.

CE QUE CE N’EST PAS

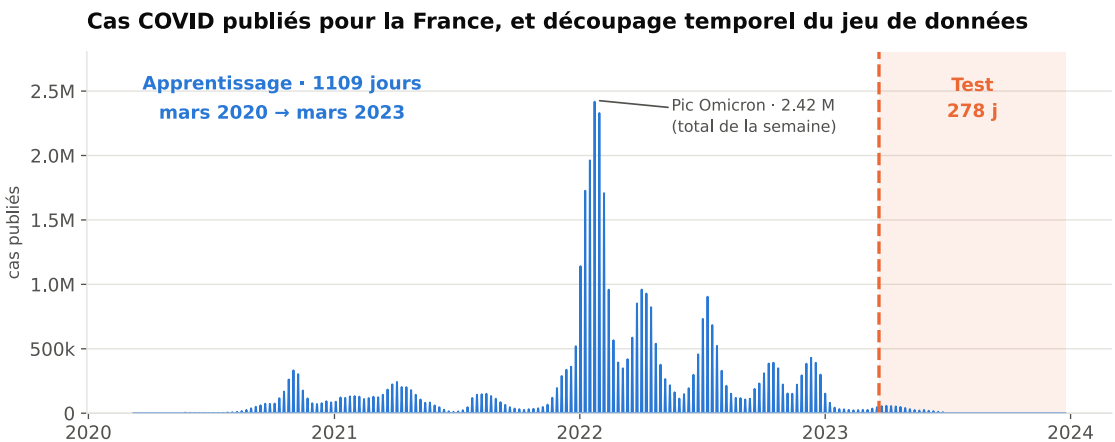
Pas un modèle épidémiologique compartimental (SIR, SEIR). Pas un modèle causal : il apprend des régularités statistiques, il n’explique pas la transmission.

De 429 435 lignes mondiales à 1 387 jours français

Six colonnes retenues sur l'export public d'OWID, filtrées sur la France, triées chronologiquement, puis bornées à la fenêtre d'étude.

`new_cases`	cas publiés — la cible
`new_deaths`	décès publiés
`people_vaccinated`	personnes vaccinées, cumul
`stringency_index`	sévérité des restrictions, 0–100

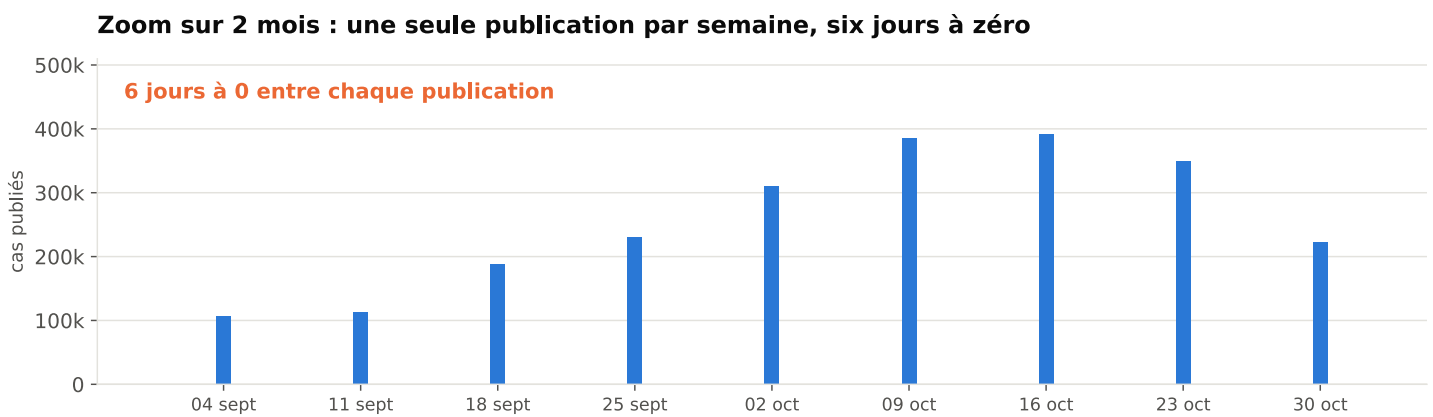
Valeurs manquantes comblées par 0, cas négatifs ramenés à 0 (corrections rétroactives des bulletins).



Fenêtre d'étude : 8 mars 2020 → 24 décembre 2023. Le découpage est strictement chronologique — le modèle n'apprend que sur le passé.

88 % des journées valent zéro

La France ne publie ses chiffres qu'une fois par semaine. Les six autres jours sont absents du fichier source — et le prétraitement les remplace par 0. La série n'est donc pas quotidienne : c'est un **total hebdomadaire posé sur un seul jour**, entouré de six zéros.



1 224 / 1 387

JOURS À ZÉRO, JEU COMPLET

265 / 278

JOURS À ZÉRO, JEU DE TEST

13

VRAIES PUBLICATIONS À PRÉDIRE

Onze variables construites par une fenêtre glissante

ENTRÉES — 7 JOURS CONNUS



CONTEXTE DU JOUR J

`new_deaths` `people_vaccinated` `stringency_index`

Sept décalages, une moyenne mobile, trois variables de contexte — onze entrées, aucune information postérieure au jour J.

LE DÉCALAGE DE LA CIBLE

```
df["target"] = df["new_cases"].shift(-7)
```

Chaque ligne apprend à relier son passé immédiat à ce qui arrivera une semaine plus tard.

APRÈS NETTOYAGE

1 387 lignes exploitables sur 1 401 jours de calendrier. Quatorze journées, en tête et en queue de série, sont écartées : les décalages ou la cible y sont incalculables.

Une forêt aléatoire, entraînée dans l'ordre du temps

CHOIX	VALEUR
`RandomForestRegressor`	scikit-learn
`n_estimators`	100 arbres
`random_state`	42 — résultats reproductibles
`profondeur`	non bornée (défaut)

Un ensemble d'arbres capte des seuils et des interactions non linéaires sans exiger de normalisation ni d'hypothèse de distribution — un point de départ solide sur onze variables hétérogènes.

LE POINT CRITIQUE : PAS DE MÉLANGE

Le découpage est **chronologique**, jamais aléatoire. Mélanger les lignes ferait apprendre au modèle des jours postérieurs à ceux qu'il doit prédire : les scores s'envoleraient et le modèle ne vaudrait rien en production.

1 109 jours

APPRENTISSAGE

mars 2020 → mars 2023

278 jours

TEST

mars → déc. 2023

Le modèle bat nettement les références naïves

943 cas

MAE — ERREUR ABSOLUE MOYENNE

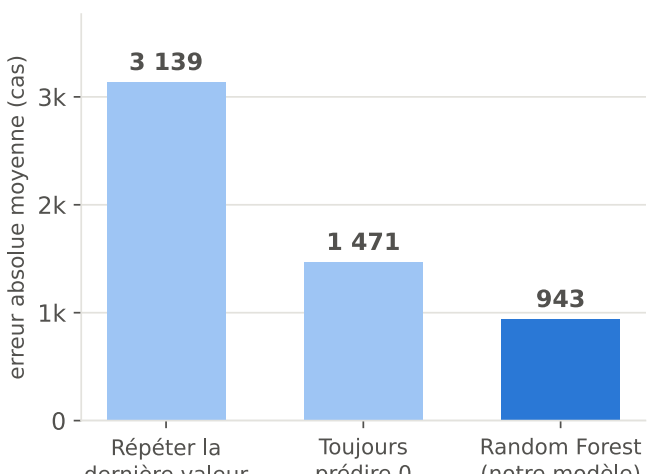
4 262 cas

RMSE — ERREUR QUADRATIQUE

0,68

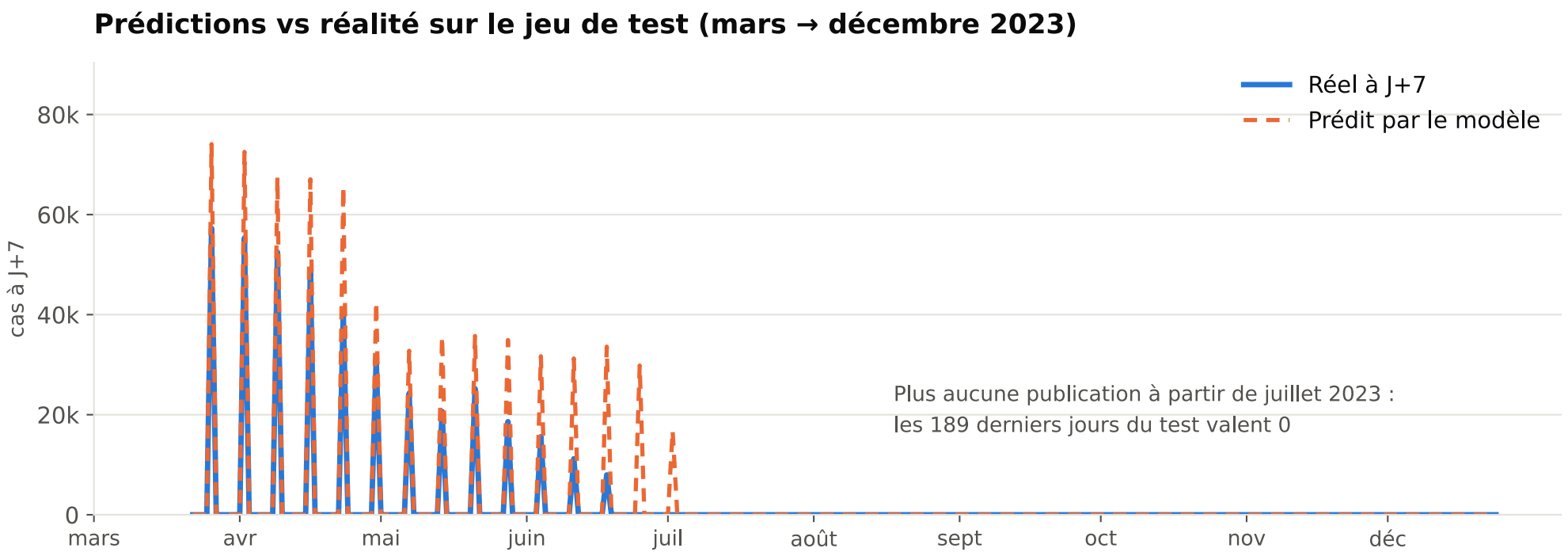
R^2 — VARIANCE EXPLIQUÉE

MAE : le modèle face à deux références simples



- **−70 % d'erreur** face à la référence « répéter la dernière valeur connue » (3 139 cas de MAE).
- **−36 % d'erreur** face à la référence « toujours prédire zéro » (1 471 cas) — la barre la plus exigeante, vu la structure des données.
- Le **rapport RMSE / MAE de 4,5** est le vrai signal à lire : il dit que l'erreur n'est pas répartie uniformément. La diapositive 09 montre où elle se concentre.

Le modèle a compris le rythme, pas encore l'amplitude

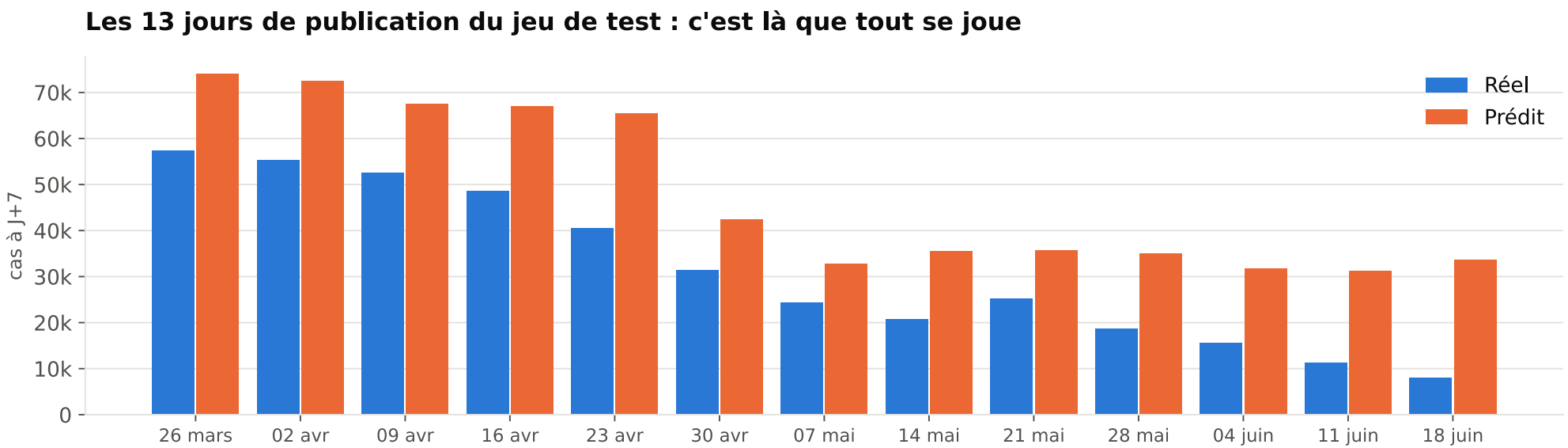


Le rythme est juste. Les pics prédits tombent exactement les jours de publication : la périodicité hebdomadaire est parfaitement restituée.

L'amplitude est surestimée. Sur les treize publications, la prédiction dépasse le réel à chaque fois — le modèle reste ancré sur les niveaux, plus élevés, vus à l'entraînement.

Puis plus rien. La France cesse toute publication après le 18 juin 2023 : les 189 derniers jours du test sont des zéros que le modèle prédit sans effort.

Toute l'erreur se concentre sur treize journées



176 cas

MAE SUR LES 265 JOURS SANS PUBLICATION

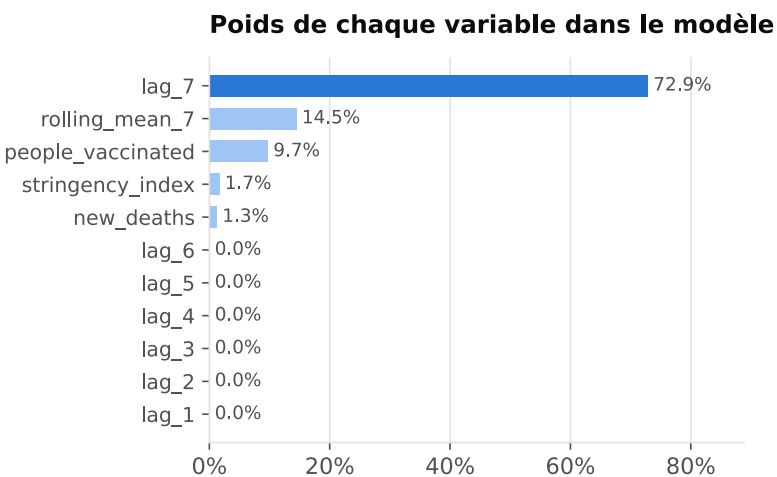
16 576 cas

MAE SUR LES 13 JOURS DE PUBLICATION

77 %

PART DE L'ERREUR QUADRATIQUE DUE À CES 13 JOURS

Voilà pourquoi la MAE est flatteuse : moyennée sur 278 jours dont 265 sont triviaux, elle dilue l'erreur réelle d'un facteur dix-sept.

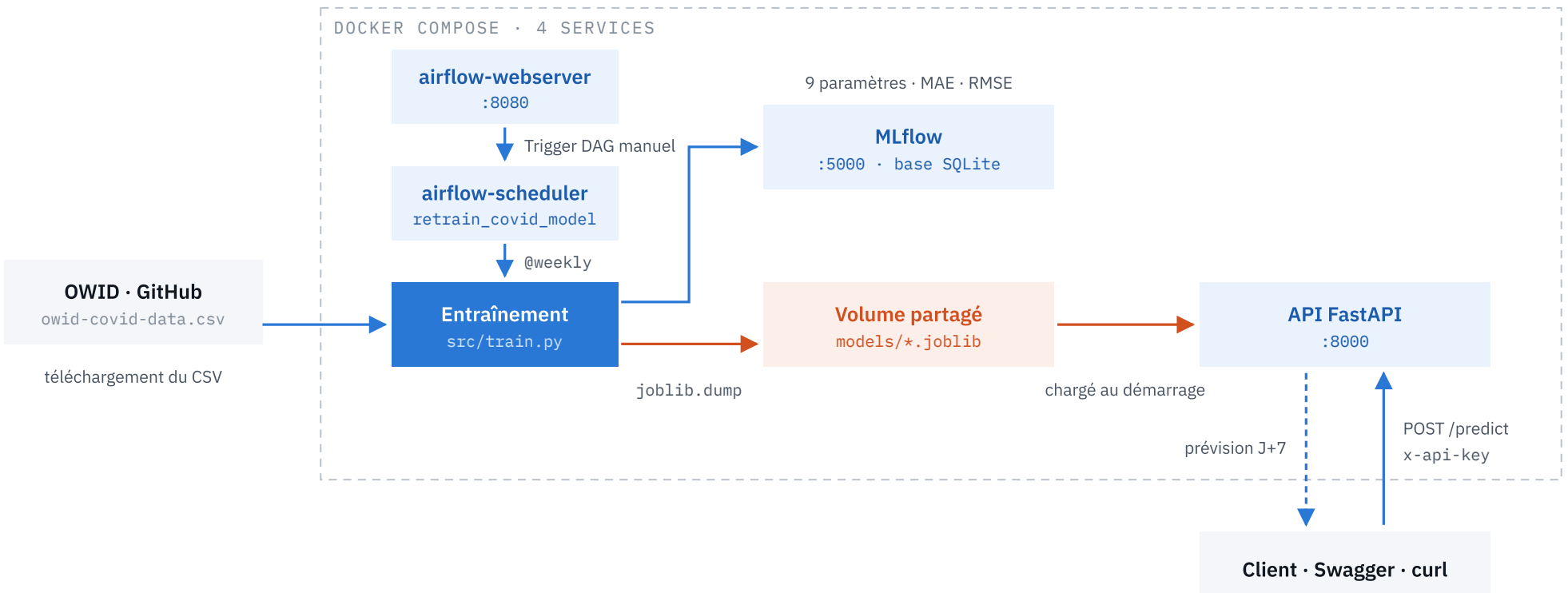


Le modèle a redécouvert seul la cadence hebdomadaire

- **lag_7 pèse 72,9 %** à lui seul : la dernière publication connue, exactement sept jours plus tôt, est de loin le meilleur prédicteur.
- **lag_1 à lag_6 pèsent 0,0 %** — pas « presque zéro », zéro exact. Ces six variables valent 0 six fois sur sept : elles ne portent strictement aucune information.
- La **vaccination cumulée (9,7 %)** joue en réalité le rôle d'un indicateur de calendrier : elle croît de façon monotone et situe la date dans l'épidémie.

Ce graphique est le meilleur diagnostic du projet : il confirme, depuis l'intérieur du modèle, le défaut de structure identifié à la diapositive 04.

Quatre services, un volume partagé, une boucle fermée



La boucle est fermée : chaque réentraînement écrit un nouveau modèle sur le volume et un nouveau run dans MLflow, sans toucher au code de l'API.

Trois points d'entrée, une clé, une réponse

ENDPOINT	RÔLE
`GET /`	message d'état
`GET /health`	modèle chargé, pays, horizon, MAE, RMSE
`POST /predict`	prévision — protégé par clé
`GET /docs`	documentation Swagger générée

- **Validation Pydantic** des onze champs : un type erroné est rejeté avant d'atteindre le modèle.
- **Clé API** en en-tête `x-api-key` ; absente ou incorrecte, la requête reçoit un `401`.
- **Chargement unique** du modèle au démarrage du service, pas à chaque appel.

REQUÊTE

```
POST /predict
x-api-key: mlops-covid-secret-key

{
  "lag_1": 1200,  … "lag_7": 1500,
  "rolling_mean_7": 1300,
  "new_deaths": 25,
  "people_vaccinated": 40000000,
  "stringency_index": 45.5
}
```

RÉPONSE

```
200 OK

{ "prediction_new_cases_h_plus_7": 5300.66 }
```

Chaque entraînement laisse une trace, et se relance sans intervention

MLFLOW — LA MÉMOIRE DES EXPÉRIENCES

- Expérience `covid_prediction_experiment`, stockée en SQLite, consultable sur `:5000`.
- **9 paramètres** journalisés par run : pays, fenêtre de dates, horizon, graine, type de modèle, nombre d'arbres, taille du train, liste des variables.
- **2 métriques** : MAE et RMSE — de quoi comparer deux versions sans relire le code.

3 runs enregistrés à ce jour, aux métriques identiques : la graine fixée rend l'entraînement parfaitement reproductible.

AIRFLOW — LA BOUCLE DE RÉENTRAÎNEMENT

- DAG `retrain_covid_model`, planifié `@weekly` — la même cadence que la publication des données.
- Une tâche `BashOperator` relance la chaîne complète : téléchargement, prétraitement, variables, entraînement, journalisation.
- `retries=1` avec cinq minutes d'attente ; `catchup=False` pour ne pas rattraper l'historique au premier démarrage.

Le déclenchement manuel reste possible depuis l'interface — utile pour une démonstration ou après un correctif.

La chaîne en fonctionnement, du démarrage au réentraînement

01 **`docker compose up -d`**
Les quatre conteneurs démarrent ensemble. `docker ps` les affiche : API, MLflow, webserver et scheduler Airflow.

02 **Swagger sur `:8000/docs`**
L'API se documente seule : les onze champs attendus, leurs types et un exemple pré-rempli.

03 **`GET /health`**
Le modèle répond avec sa propre carte d'identité : pays, horizon, MAE et RMSE embarqués dans le fichier `.joblib`.

04 **`POST /predict` sans clé → 401**
La protection se vérifie en une requête : sans en-tête `x-api-key`, rien ne passe.

05 **`POST /predict` avec la clé → 200**
Sur la charge utile d'exemple, le modèle renvoie **5 300,66 cas** à J+7.

06 **MLflow sur `:5000`**
Les runs côte à côte, avec leurs neuf paramètres et leurs deux métriques : on compare deux versions sans relire une ligne de code.

07 **Airflow sur `:8080` → Trigger DAG**
`retrain_covid_model` se déclenche à la main ; la tâche passe au vert en quelques minutes.

08 **Retour sur MLflow**
Un run de plus est apparu, et le volume partagé contient un nouveau modèle. La boucle s'est refermée sans toucher au code de l'API.

PLAN DE REPLI

Si un service tarde à démarrer, le récit se déroule à l'identique sur les captures de secours et sur le rapport `report.html`, qui montre les trois tests au vert.

Ce qui est vérifié, et ce qui ne l'est pas encore

3 / 3

TESTS PYTEST RÉUSSIS

0

ÉCHEC

TEST	ATTENDU
`/health`	200 · modèle chargé
`/predict sans clé`	401
`/predict avec clé`	200 · prévision

Les tests s'exécutent contre l'application réelle via le `TestClient` de FastAPI, modèle chargé compris.

CONTENEURISATION

Trois images (API, MLflow, Airflow) et quatre services orchestrés par un seul `docker compose up`. Trois volumes assurent la persistance : modèle entraîné, base MLflow, métadonnées Airflow.

ANGLES MORTS ASSUMÉS

- Aucun test unitaire sur le prétraitement, les variables et l'entraînement — seule l'API est couverte.
- Pas de test d'intégration bout en bout sur la chaîne de réentraînement.

Six limites que nous assumons

01 **La cadence hebdomadaire n'est pas traitée.** Travailler sur une série « quotidienne » composée à 88 % de zéros rend les métriques globales flatteuses et peu informatives.

02 **Six variables sur onze sont inertes.** lag_1 à lag_6 ne portent aucune information : le modèle en supporte le coût sans aucun bénéfice.

03 **Surestimation systématique en décrue.** Sur les treize publications testées, l'erreur relative moyenne atteint 82 %.

04 **Le modèle n'est pas versionné.** L'appel au registre MLflow est commenté dans le code : impossible de promouvoir ou de revenir en arrière proprement.

05 **Le secret est en clair.** La clé API figure dans `docker-compose.yml` et dans le dépôt.

06 **Aucune surveillance en production.** Ni détection de dérive des données, ni alerte si la qualité des prévisions se dégrade.

Ces limites ne sont pas des oublis : elles ont été identifiées en analysant les résultats, et c'est cette analyse qui a produit les diapositives 04, 09 et 10.

La chaîne tient ; c’est le signal qu’il faut maintenant redresser

SUR LA MODÉLISATION

- **Rééchantillonner en pas hebdomadaire.** Une série de 201 points denses remplacerait 1 387 points dont 88 % de zéros — les décalages redeviendraient tous porteurs d’information.
- **Évaluer sur les seuls jours de publication**, et publier systématiquement l’écart à une référence naïve.
- **Valider par TimeSeriesSplit** plutôt que par un unique découpage, et comparer la forêt à une régression régularisée et à un gradient boosting.

SUR L’INDUSTRIALISATION

- **Activer le registre de modèles MLflow** avec des étapes Staging / Production et un retour arrière en une commande.
- **Superviser avec Prometheus et Grafana** : latence et taux d’erreur de l’API, MAE glissante recalculée à chaque publication, et une alerte dès qu’elle dépasse son seuil.
- **Sortir les secrets du dépôt** et surveiller la dérive des données entrantes.
- **Étendre les tests** au prétraitement et à la construction des variables, puis les brancher sur une intégration continue.

L’infrastructure MLOps est en place et fonctionne de bout en bout. Le travail restant porte désormais sur la donnée et sur l’évaluation — c’est-à-dire sur ce qui fait la valeur d’une prévision.